

День 7 • 🤖 Создание первого AI-агента, знакомство с LangChain

📋 Обсуждаемые темы:

- 🤔 Что такое AI-агент? (5 минут)
- 📋 Примеры AI-агентов (5 минут)
- 🎯 Зачем нужны AI-агенты? (5 минут)
- 🔗 Что такое LangChain? (10 минут)
- 🏗️ Архитектура простого AI-агента (15 минут)

🤔 Что такое AI-агент?

AI-агент — это автономная или полуавтономная система, которая:

- 📥 получает входные данные (вопрос, событие, файл),
- 🔄 обрабатывает их с помощью LLM или логики сценариев,
- 🤔 принимает решения или предлагает действия,
- 🛠️ может выполнять действия в окружении (через API , CLI , GitHub и пр.)

Примеры AI-агентов

-  Техподдержка на базе `RAG` / `Retrieval Augmented Generation`
-  Агент-ревьюер для анализа кода или `PR`
-  **Cursor Agent** — агент прямо в IDE, пишет и рефакторит код
-  **GitHub Copilot Workspace** — агент для работы с задачами в репозитории
-  **Devin / OpenHands** — автономные агенты-разработчики

Зачем нужны AI-агенты?

- ⚡ Снижают нагрузку на команду
- 🚀 Повышают скорость обработки типовых задач
- 🔄 Уменьшают рутину в работе разработчиков, тимлидов и менторов

Что такое LangChain?

LangChain — фреймворк для построения приложений на базе LLM :

-  **Chains** — цепочки шагов: промпт → модель → парсер
-  **Memory** — контекст разговора между запросами
-  **Tools** — подключение внешних инструментов (API , поиск , код)
-  **Agents** — LLM сам решает, какой инструмент вызвать

LangChain: экосистема 2025–2026

Пакет	Назначение
<code>langchain</code>	Базовые абстракции и цепочки
<code>langchain-community</code>	Интеграции (GigaChat, OpenAI, Chroma...)
<code>langgraph</code>	Граф-агенты с состоянием, циклами
<code>langsmith</code>	Трассировка, отладка, мониторинг

LangGraph — актуальный подход для сложных агентов.

LangChain: простой агент с инструментом

```
from langchain_gigachat import GigaChat
from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.tools import tool

@tool
def get_pr_diff(pr_number: int) -> str:
    """Возвращает diff для PR по номеру."""
    # вызов GitHub API
    ...

llm = GigaChat(credentials="...", verify_ssl_certs=False)
agent = create_tool_calling_agent(llm, [get_pr_diff], prompt)
executor = AgentExecutor(agent=agent, tools=[get_pr_diff])
```



AI-агенты для анализа PR с использованием GigaChat API



Источник данных (GitHub PR)



Обработка текста



GigaChat



Ответ / отчёт



Архитектура простого AI-агента: этапы

1.  Получение данных по API (PR title , body , diff)
2.  Формирование промпта и запрос в LLM
3.  Обработка ответа и его форматирование
4.  Возврат пользователю (через консоль, бота или комментарий к PR)

★ Почему GigaChat?

-  Поддержка русского и английского языков
-  Обучен на разработческом контенте: код, документация, команды
-  Можно использовать в приватных или защищённых средах
-  Отлично справляется с код-ревью, структурой проектов, пояснениями

-  Сделать агента активным: создаёт PR , комментирует, ставит метки
-  Подключить источники: Jira , Confluence , внутренние документы
-  Перейти на LangGraph для сложных сценариев с ветвлением и циклами
-  Multi-agent: несколько агентов с разными ролями работают вместе

AI-агент — это не абстракция, а конкретный скрипт, обёрнутый в умную логику

- ★ GigaChat отлично подходит для первых экспериментов: он работает на русском, разбирается в коде и легко подключается через API
- 🚀 Даже базовый агент может принести пользу команде, снижая рутину (например: улучшая качество pull request'ов)

Вы получили новые инструменты — но опыт начинается с практики... 

 **Исследуйте** — пробуйте код, экспериментируйте, ошибайтесь

 **Углубляйтесь** — каждая тема открывает новые возможности

 **Создавайте** — даже маленький проект лучше теории

Это невозможно, пока вы это не сделаете!

Успехов!